

Library Management System

with Book Recommendation & Late-Return Risk Prediction | SE1 Project Documentation

1. Project Overview

A full-stack Library Management System for handling books, members, and borrowing transactions, enhanced with two applied machine learning features: a book recommendation engine that personalizes suggestions for each member, and a late-return risk model that flags borrowers likely to return a book past its due date. The system covers the complete lifecycle of a library workflow — cataloging, borrowing, returns, fines, and reservations — while the ML layer adds predictive intelligence on top of standard CRUD operations.

Objectives

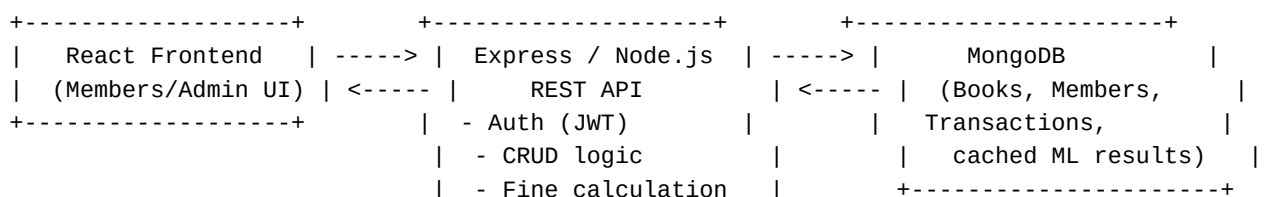
- Build a complete, functional library management system covering catalog, membership, and transaction management
- Apply machine learning to solve two real operational problems: book discovery and overdue-return prevention
- Demonstrate a clean separation between the main application (MERN) and the ML service (Python), a common real-world architecture pattern
- Deliver an intuitive, role-based UI for both librarians/admins and members

2. Tech Stack

- **MongoDB** — primary database for books, members, transactions, and cached ML outputs
- **Express.js** — REST API layer, business logic, authentication
- **React** — frontend UI for members and librarians/admins
- **Node.js** — backend runtime
- **Python (Flask)** — dedicated microservice for the ML models (recommendation + late-return risk)
- **scikit-learn / XGBoost** — model training for late-return classification
- **pandas / TF-IDF (scikit-learn)** — feature preparation for the recommendation engine

3. System Architecture

The system follows a layered architecture with the ML component isolated as its own service. This keeps the MERN team and the ML team working independently, and means the core app keeps functioning even if the ML service is temporarily down.



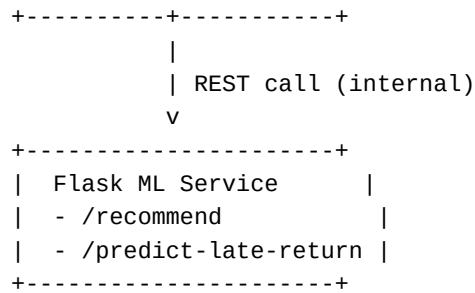


Figure 1. High-level system architecture

Request flow example (borrowing a book)

- Librarian submits a borrow request via React → Express creates a Transaction record in MongoDB
- Express calls the Flask service's /predict-late-return endpoint with the member's and book's features
- Flask returns a risk score → Express stores it on the transaction and displays it to the librarian
- Separately, when a member browses the catalog, React calls Express → Express fetches cached recommendations from MongoDB (refreshed periodically by the Flask service, not computed live on every request)

4. Database Design

MongoDB collections below. Relationships are maintained via referenced ObjectIds (Mongoose ref), consistent with standard MERN practice.

Book

Field	Type	Description
_id	ObjectId	Primary key
title	String	Book title
author	String	Author name
isbn	String	ISBN identifier
genre	String	Category/genre — used as a recommendation feature
description	String	Used for content-based recommendation (TF-IDF)
totalCopies	Number	Total copies owned
availableCopies	Number	Copies currently available
shelfLocation	String	Physical location in library

Member

Field	Type	Description
_id	ObjectId	Primary key
name	String	Full name
email	String	Login / contact
passwordHash	String	bcrypt hash
memberType	String	student / faculty — used as a risk-model feature
role	String	member / librarian / admin
lateReturnHistory	Number	Count of past late returns — risk-model feature

Transaction

Field	Type	Description
_id	ObjectId	Primary key
bookId	ObjectId (ref Book)	Borrowed book
memberId	ObjectId (ref Member)	Borrower

borrowDate	Date	Date borrowed
dueDate	Date	Date due
returnDate	Date	Actual return date (null if not yet returned)
status	String	borrowed / returned / overdue
lateReturnRiskScore	Number	Cached output from the ML service

Reservation

Field	Type	Description
_id	ObjectId	Primary key
bookId	ObjectId (ref Book)	Reserved book
memberId	ObjectId (ref Member)	Member who reserved
reservedDate	Date	Date reserved
status	String	pending / fulfilled / cancelled

Recommendation (cache)

Field	Type	Description
_id	ObjectId	Primary key
memberId	ObjectId (ref Member)	Member the recommendations are for
recommendedBooks	Array[ObjectId]	Book IDs ranked by similarity score
generatedAt	Date	Timestamp of last refresh

5. Machine Learning Components

5.1 Book Recommendation System

Content-based filtering: each book is represented as a feature vector derived from its genre, author, and description (TF-IDF vectorization). When a member views or borrows a book, the system recommends other books with high cosine similarity. Collaborative filtering (“members who borrowed this also borrowed...”) is a stretch goal once enough transaction data accumulates.

- Input features: genre, author, description text
- Technique: TF-IDF + cosine similarity (content-based)
- Output: ranked list of book IDs per member, cached in the Recommendation collection
- Refresh strategy: recompute periodically (e.g. on new borrow event) rather than on every page load

5.2 Late-Return Risk Prediction

A classification model trained on historical transaction data to flag loans likely to be returned late, so librarians can send proactive reminders.

- Input features: member type, past late-return count, book category, loan duration allowed, day of week borrowed
- Label: returned_late (yes/no), derived from returnDate vs. dueDate
- Technique: Random Forest or XGBoost classifier
- Output: risk score (low / medium / high) attached to the transaction at borrow time
- Cold-start note: real borrowing history won't exist at project start — train initially on a synthesized/simulated transaction dataset, documented clearly as such in the project report

6. Core Features Summary

- Role-based authentication (admin / librarian / member)
- Book catalog with search and filtering
- Borrow / return workflow with due-date tracking
- Automated fine calculation for overdue books
- Reservation queue for unavailable books
- **Personalized book recommendations** per member
- **Late-return risk dashboard** for librarians

7. Suggested Work Division

- **Backend/Database team** — Express routes, Mongoose schemas, authentication
- **Frontend team** — React UI, dashboards, forms
- **ML team** — dataset preparation, model training, Flask service (both endpoints)
- **Integration/QA** — connecting Flask service to Express, end-to-end testing, documentation